



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO SUPERIOR DE SAN PEDRO DE LAS COLONIAS



INGENIERÍA EN SISTEMAS COMPUTACIONALES

Fundamentos de ingeniería en software
Grupo: 5A

[U1-E02] Análisis individual de problemáticas de ingeniería de software

Caso B - C02: Plataforma de inscripción escolar

PROFESOR:
RUTH AIVI CHÁVEZ RODRÍGUEZ

Nombre del estudiante
Ortiz Reyes Karime

No. de control
241000028

San Pedro, Coahuila de Zaragoza, a 29 de Septiembre de 2026

1. Presentación del caso y consecuencias

El caso aborda la situación de una plataforma web de inscripción escolar diseñada para consultar materias, seleccionar grupos y generar comprobantes. Aunque el sistema opera correctamente en pruebas controladas con pocos usuarios experimenta una falla operativa notoria durante los días de alta demanda. Las anomalías detectadas abarcan una marcada lentitud, la duplicación de solicitudes de inscripción y la emisión de comprobantes para grupos cuyos cupos ya han sido agotados en su totalidad.

Esto desencadenan un impacto negativo en tres niveles de la institución:

- **Consecuencias para los usuarios (estudiantes):** Enfrentan incertidumbre en su avance académico al recibir comprobantes inválidos. Asimismo, sufren una mala experiencia de usuario debido a demasías en tiempos de espera y fallas en la captura de sus selecciones.
- **Consecuencias para el equipo (desarrollo y soporte):** El personal de soporte está saturado de reportes que no se pueden clasificar ni asociar a una sola versión. Por su parte, los desarrolladores carecen de diagnósticos claros y de un plan para restaurar una versión funcional y segura del sistema.
- **Consecuencias para la organización (institución):** La universidad pierde credibilidad ante la comunidad estudiantil y tiene conflictos entre departamentos por la disparidad de datos sobre los cupos reales, haciendo imposible determinar si la causa del problema es tecnológica, de datos o administrativa.

2. Identificación y análisis de cinco problemas

Problema 1: Inexistencia de pruebas de carga y rendimiento

- **Causa:** Probar el sistema únicamente con muy pocas personas antes de abrir las inscripciones para toda la escuela.
- **Consecuencia:** La plataforma se alenta y no responde a tiempo cuando se conectan muchos alumnos al mismo momento.
- **Práctica de Ingeniería de Software:** Pruebas No Funcionales (Pruebas de Carga, Estrés y Escalabilidad).
- **Evidencia del caso:** *"Durante las pruebas con pocos usuarios, todas las funciones principales responden correctamente. El día de mayor demanda, la plataforma se vuelve lenta..."*.

Problema 2: Falta de control cuando varios alumnos piden el mismo cupo al mismo tiempo

- **Causa:** No poner reglas en la base de datos para bloquear un cupo mientras alguien lo está tomando, ni evitar que un clic repetido envíe dos veces la misma solicitud.
- **Consecuencia:** Se entregan más cupos de los que existen y se duplican las solicitudes de un mismo estudiante.

- **Práctica de Ingeniería de Software:** Manejo de Transacciones y Control de Concurrencia en Bases de Datos.
- **Evidencia del caso:** "...algunas solicitudes se procesan dos veces y ciertos estudiantes reciben un comprobante, aunque el cupo ya se agotó."

Problema 3: Manejo informal y desordenado de los cambios de reglas

- **Causa:** Pedir y aceptar cambios en el sistema por medios informales como correos o mensajes sueltos.
- **Consecuencia:** No hay una lista clara de decisiones, lo que provoca confusiones sobre cómo debe funcionar la plataforma.
- **Práctica de Ingeniería de Software:** Ingeniería de Requisitos y Control de Cambios.
- **Evidencia del caso:** "Los cambios de requisitos llegan por correo y mensajes separados, sin una lista única de decisiones."

Problema 4: Carencia de plan para regresar a una versión que sí servía

- **Causa:** No tener guardadas las versiones anteriores del programa ni un plan documentado para casos de emergencia.
- **Consecuencia:** Cuando el sistema falla en el día importante, el equipo no sabe cómo regresar rápido a una versión estable que no falle.
- **Práctica de Ingeniería de Software:** Gestión de la Configuración del Software y Plan de Reversión
- **Evidencia del caso:** "No se realizan pruebas de carga ni existe un plan documentado para regresar a una versión estable."

Problema 5: Incoherencia en los datos y reportes sin organizar

- **Causa:** No tener una sola base de datos centralizada y no conectar el sistema de soporte con el sistema principal.
- **Consecuencia:** Es imposible saber si el problema viene del código, de los datos o de la administración, y las áreas terminan alegando con datos diferentes
- **Práctica de Ingeniería de Software:** Base de Datos Unificada y Seguimiento Organizado de Fallas
- **Evidencia del caso:** "Soporte recibe reportes, pero no los clasifica por prioridad ni puede relacionarlos con una versión específica. Cada área utiliza cifras diferentes para explicar los cupos..."

3. Tres acciones prioritarias

- **Acción 1: Arreglar la base de datos para que no dé cupos de más**
 - **Orden:** 1.^º (Prioridad Alta).

- **Justificación:** Es lo más importante porque evita que se dupliquen inscripciones y garantiza que los cupos sean reales.
- **Costo o limitación:** Requiere pausar otras tareas para corregir cómo se guardan las inscripciones en el sistema.
- **Comprobación:** Simular con un programa que 100 alumnos intenten agarrar el último cupo al mismo segundo, comprobando que solo 1 lo consiga y a los otros 99 les avise que ya se agotó.
- **Acción 2: Hacer pruebas con tráfico simulado y tener un plan de emergencia**
 - **Orden:** 2.º (Prioridad Media).
 - **Justificación:** Sirve para saber cuántos alumnos aguanta la plataforma antes de abrirla al público y tener cómo solucionar las cosas rápido si algo sale mal.
 - **Costo o limitación:** Toma tiempo preparar las pruebas e implica un costo por usar servidores de prueba.
 - **Comprobación:** Probar la plataforma con herramientas que simulen a miles de usuarios conectados y verificar que responda en menos de 2 segundos. También probar que se pueda regresar a la versión anterior en menos de 10 minutos si ocurre una falla.
- **Acción 3: Usar un solo lugar para ver los cupos y para anotar los cambios**
 - **Orden:** 3.º (Prioridad Normal).
 - **Justificación:** Evita que los departamentos tengan números distintos y asegura que todos los cambios se aprueben en un solo lugar.
 - **Costo o limitación:** El personal tendrá que acostumbrarse a usar una sola herramienta para hacer solicitudes y reportar fallas.
 - **Comprobación:** Revisar que todos los departamentos vean exactamente el mismo número de cupos y que cualquier cambio esté registrado en la plataforma de trabajo.

Reflexión personal

Con este caso me doy cuenta de que hacer software no es solo programar algo que funcione en mi computadora con un solo usuario. Para mis futuros proyectos quisiera implementar lo siguiente:

- No confiarme solo porque el programa sirva cuando lo pruebo yo solo; haré pruebas imaginando a muchos usuarios conectados al mismo tiempo.
- Estar preparado por si falla: Siempre guardaré copias de las versiones que sí funcionan por si necesito regresar a una versión segura de inmediato.

Fuentes consultadas y declaración de uso de IA

Fuentes consultadas: Material del curso: *Casos para U1-E02: Análisis individual de problemáticas de ingeniería de software (Caso B: C02 Plataforma de inscripción escolar)*.
Asignatura Fundamentos de Ingeniería de Software, Google Classroom.

Declaración del uso de Inteligencia Artificial:

- Declaro que utilicé la herramienta de Inteligencia Artificial (Gemini) únicamente como apoyo para organizar las secciones y revisar la redacción. El análisis de los problemas y las soluciones propuestas fueron realizados por mí conforme a lo visto en clase.